



WEEK 4 – DATA 5600

Spring 2026

Pol Monné Parera (L30170743)
pmonneparera@lewisu.edu

1. Technical Description

1.1 Restricted Boltzmann Machine (RBM)

A Restricted Boltzmann Machine (RBM) is a two-layer undirected generative neural network first introduced by Smolensky in 1986. This type of neural network is commonly used for pre-training deep networks.

An RBM is composed of a visible layer and a hidden layer, with symmetric weighted connections between them but no intra-layer connections (from this comes the “restricted” naming). The model is energy based.

1.2 MNIST Dataset

MNIST (Modified National Institute of Standards and Technology) is a well utilized dataset containing 70,000 grayscale images of handwritten zero to nine digits, each picture with a size of 28 by 28 pixels. This dataset is well-suited for an RBM as pixel intensities can be scaled to $[0, 1]$ and binarized.

1.3 Logistic Regression

Logistic regression is a supervised machine learning algorithm used for binary classification, predicting the probabilities of an outcome based on independent variables. It uses a sigmoid function to map predicted values to probabilities between 0 and 1, creating an S-shaped curve. In this project it serves as the downstream classifier in both the baseline pipeline (raw pixels as input) and the RBM pipeline (hidden activations as input).

2. Design of the Algorithms

2.1 Dataset

The MNIST dataset is downloaded directly from *tf.keras.datasets.mnist* and pre-split into 60,000 training and 10,000 test images. Later, both raw and RBM-transformed versions of train and test are passed to Logistic Regression.

2.2 Preprocessing Steps

1. **Flatten:** reshape each 28×28 image into a 784-dimensional vector.
2. **Normalize:** divide pixel values by 255 to map them to $[0, 1]$.
3. **Binarize:** apply threshold of 0.5. Pixels above become 1, others become 0.

2.3 Pseudocode

// PREPROCESSING

LOAD MNIST (60k train, 10k test)

FOR each image: FLATTEN → NORMALIZE → BINARIZE with threshold 0.5

// TRAINING (CD-1)

FOR epochs:

 FOR batches:

 hidden ← from input

 reconstruction ← from hidden

 hidden2 ← from reconstruction

 update parameters

// FEATURE EXTRACTION

x_train_rbm ← hidden representation of training data # 60k × 256

x_test_rbm ← hidden representation of test data # 10k × 256

// CLASSIFICATION

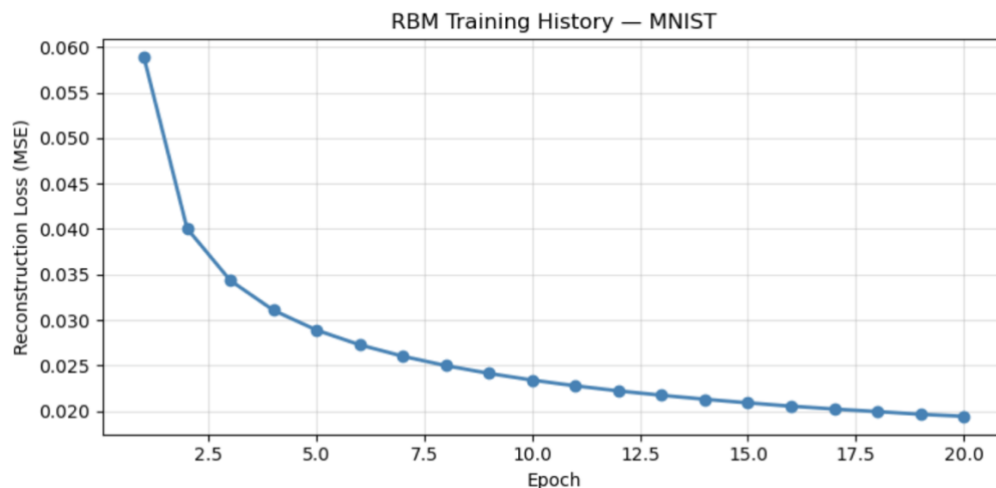
FIT LogisticRegression on (x_train_raw, y_train) → predict on x_test_raw

FIT LogisticRegression on (x_train_rbm, y_train) → predict on x_test_rbm

COMPARE accuracy of both pipelines

3. Results of the Algorithm

3.1 Training Curve

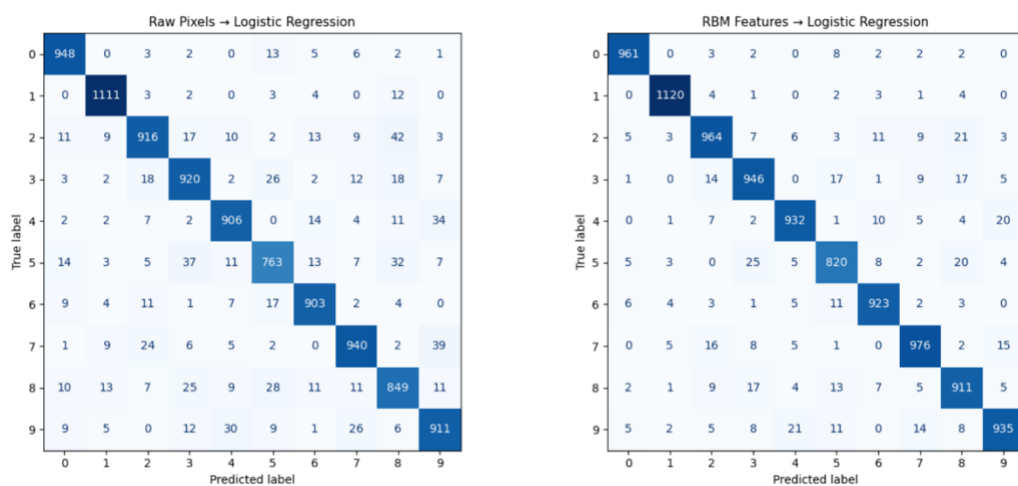


3.2 Reconstruction Quality

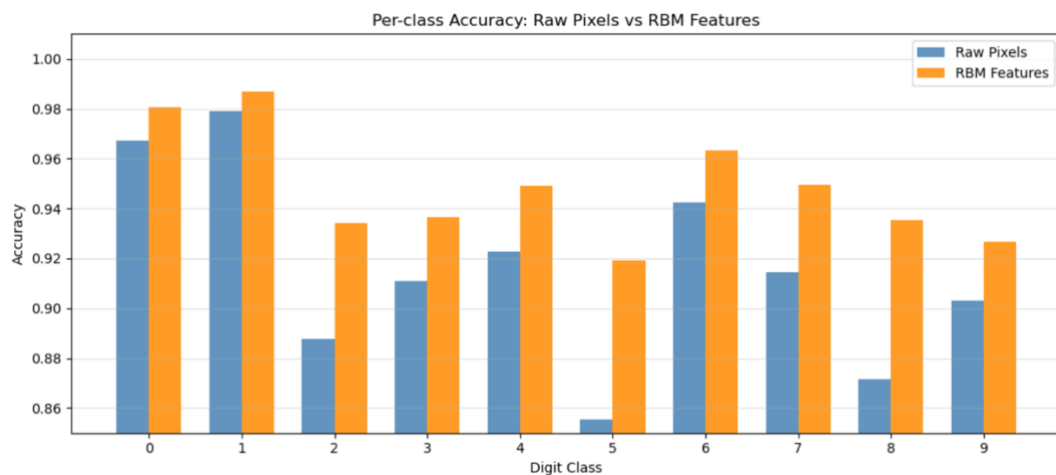
In the image below the first row is the original image, the second the reconstruction, the third the binarized reconstruction and the fourth the difference map.



3.3 Confusion Matrices



3.4 Per-class Accuracy



3.5 Classification Accuracy

Pipeline	Feature Dimensions	Test Accuracy
Raw Pixels Logistic Regression	784	91.67%
RBM (256 units) Logistic Regression	256	94.88%

4. Analysis of the Results

4.1 Did I Obtain What I Expected?

The outcome obtained was largely consistent with the expectations. I had anticipated that the RBM would learn meaningful representations of the images, which would equal or improve the classification accuracy obtained with the raw pixels. The accuracy comparisons reveal that the RBM features achieve a slightly better classification accuracy than raw pixels despite using only 256 features instead of 784.

4.2 Surprises

The biggest surprise observe is the per-class accuracy improvement is uneven. Classes such as zero and one show the highest accuracy on both pipelines, while other classes like two, five and eight are the ones that appear to benefit the most from the RBM representation. This suggests that the hidden units can capture features that are especially useful for visually similar pairs.

4.3 Conclusions

The RBM feature extraction successfully obtained relevant features from the raw MNIST digits which were useful to improve performance of the base Logistic Regression classification model. The results are consistent with the original motivation of RBMs as pre-training tools for deep networks, showing that learned representations provide a better initialization than random weights.

Some possible future improvements could include the following:

1. Stack two RBMs to form a Deep Belief Network (DBN) and compare representation quality and accuracy.
2. Increase hidden units to 512 and compare reconstruction fidelity.
3. Compare RBM features to those produced by a shallow autoencoder of equivalent parameter count.

5. Program Listings

```
def preprocess(x, threshold=0.5):
    """
    Flatten, normalize to [0,1], then binarize pixel values.
    Returns float32 array of shape (N, 784).
    """
    x = x.astype("float32") / 255.0      # normalize
    x = x.reshape(len(x), -1)          # flatten
    x = (x > threshold).astype("float32") # binarize
    return x

x_train = preprocess(x_train_raw)
x_test = preprocess(x_test_raw)

print("Preprocessed train shape:", x_train.shape)
print("Preprocessed test shape: ", x_test.shape)
print("Unique pixel values:", np.unique(x_train))
```

Figure 1 Preprocess of MNIST digit data

```
class RBM(tf.Module):
    def __init__(self, n_visible: int, n_hidden: int, learning_rate: float = 0.01, name=None):
        super().__init__(name=name)
        self.n_visible = n_visible
        self.n_hidden = n_hidden
        self.lr = learning_rate

        self.W = tf.Variable(tf.random.normal([n_visible, n_hidden], stddev=0.01), name="W")
        self.b_v = tf.Variable(tf.zeros([n_visible]), name="b_v")
        self.b_h = tf.Variable(tf.zeros([n_hidden]), name="b_h")

    def hidden_probs(self, v):
        return tf.nn.sigmoid(tf.matmul(v, self.W) + self.b_h)

    def visible_probs(self, h):
        return tf.nn.sigmoid(tf.matmul(h, tf.transpose(self.W)) + self.b_v)

    def sample(self, probs):
        return tf.cast(tf.random.uniform(tf.shape(probs)) < probs, tf.float32)

    def contrastive_divergence(self, v0):
        h0_prob = self.hidden_probs(v0)
        h0_sample = self.sample(h0_prob)

        v1_prob = self.visible_probs(h0_sample)
        v1_sample = self.sample(v1_prob)
        h1_prob = self.hidden_probs(v1_sample)

        bs = tf.cast(tf.shape(v0)[0], tf.float32)
        dW = (tf.matmul(tf.transpose(v0), h0_prob) -
              tf.matmul(tf.transpose(v1_sample), h1_prob)) / bs
        dbv = tf.reduce_mean(v0 - v1_sample, axis=0)
        dbh = tf.reduce_mean(h0_prob - h1_prob, axis=0)

        self.W.assign_add(self.lr * dW)
        self.b_v.assign_add(self.lr * dbv)
        self.b_h.assign_add(self.lr * dbh)

        return tf.reduce_mean(tf.square(v0 - v1_prob))

    def reconstruct(self, v):
        h = self.hidden_probs(v)
        return self.visible_probs(h)

    def transform(self, v):
        return self.hidden_probs(v).numpy()
```

Figure 2 RBM class and methods definition

```
print("Training Logistic Regression on RBM-learned features...")
lr_rbm = LogisticRegression(max_iter=2000, solver="saga", random_state=42, n_jobs=-1)
lr_rbm.fit(x_train_rbm, y_train)

y_pred_rbm = lr_rbm.predict(x_test_rbm)
acc_rbm = accuracy_score(y_test, y_pred_rbm)

print(f"\nAccuracy (RBM Features): {acc_rbm * 100:.2f}%")
```

Figure 3 Logistic Regression

References

TensorFlow documentation: `tf.keras.datasets.mnist`.

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist